

Software Defect Prediction based on Conditional Random Field in Imbalance Distribution

Chunhui Yang

¹Wuhan University of Technology

²Software Quality Engineering Research Center,
CEPREI

Guangzhou, China

yangch@ceprei.com

Jianwen Xiang

Wuhan University of Technology

Wuhan, China

jwxiang@whut.edu.cn

Yan Gao*

¹South China University of Technology

²Software Quality Engineering Research Center,
CEPREI

Guangzhou, China

thegoldfishwang@163.com

Lixin Liang

Faulty of Compute Science,

Guangdong University of Technology,

Guangzhou, China

gdut_lixin@163.com

Abstract—To contribute software testing, and save testing costs, a wide range of machine learning approaches have been studied to predict defects in software modules. Unfortunately, the imbalanced nature of this type of data increases the learning difficulty of such a task. In this paper, we present UCRF, a method based on undersampling technique and conditional random field (CRF) for software defect prediction in imbalance distribution. In our proposed method, firstly, we leverage meanshift clustering method to reduce the samples of majority class for balancing the train data set. Secondly, we propose to apply CRF model in the above balanced train data set because the CRF model can handle complex features without any change in training procedure. Interestingly, we find that the UCRF method achieves much better final results than the other approach as shown in the software defect data classification task.

Keywords—Software defect prediction, imbalance distribution, conditional random field, meanshift clustering method.

I. INTRODUCTION

A software defect is an error, flaw, mistake, failure, or fault in the computer program or system that produces incorrect or unexpected results, or causes it to behave in unintended way [2]. Software defect prediction (SDP) is the process of locating defective modules in software. It helps to improve software quality and testing efficiency by constructing predictive models from code attributes to enable a timely identification of fault-prone modules.

For looking for a high-performance defect predictor, researchers have been working on the choice of static attributes and effective learning algorithms since the 1990s. Moreover, seeking a better subset of attributes and choosing a good learning algorithm was shown to be at least equally important to the final performance [1]. Various statistical and machine learning methods have been investigated for SDP, among which Naive Bayes [1] and Random Forest [2], [3]

were shown to have relatively high, stable performance [2], [4]. Nevertheless, an important feature of the SDP problem have not been taken into consideration by these studies, i.e., the highly imbalanced nature between the defect and non-defect classes of the data set. In most cases, the collected train data contains much more non-defective modules (majority) than defective ones (minority).

The imbalanced distribution is a primary factor accounting for the poor performance of certain machine learning methods, especially on the minority class [4]. Class imbalance learning is a growing research direction in machine learning that aims to better handle this kind of problem [6]. It contains a number of data-level and algorithm-level techniques. Several researchers took notice of the negative effect of class imbalance on SDP and considered utilizing class imbalance learning technologies to improve the performance of their predictors recently [5], [7], [8]. However, it is still indistinct what extent class imbalance learning can contribute to SDP, and how to make better use of it to improve SDP.

In this paper, UCRF (Undersample Conditional Random Field), a method based on undersampling technique and CRF Model is introduced to improve software defect prediction in imbalance distribution by balancing original data set and incorporating complex features without any change in training procedure. Our method contains two part: (1) due to the imbalanced distribution is a main factor accounting for the poor performance of certain machine learning methods, we propose to balance the train data set by meanshift clustering method. That is to reduce the samples of majority class so as to balance the train data set. As a result, we can acquire a balanced data set. (2) It is different from the previous work [1], [2], [3], [4], we propose to adopt CRF model which can have the ability to handle complex features without any change in training procedure in the above balanced data set. We note that the CRF model is easily extensible to

recognition since it is a state and label sequence modeling technique. We show that UCRF can easily be trained using the simple direct optimization technique of stochastic gradient descent. We demonstrate the results on the software defect data classification task and show that UCRF outperforms other approaches such as [6], [7], [8]. In fact, UCRF results on this task are the best single classifier results known to us.

The remainder of this paper is structured as follows. In Section 2, related work is briefly summarized. A formalization description of leveraging undersampling technique to balance data set, our solution of using CRF model and the work flow are described in detail in Section 3. Experiments results and analysis are reported in Section 4 and Section 5 concludes this paper with some future work.

II. RELATED WORK

This section introduces the current research progress in software defect prediction.

For SDP, because the nature of the problem, the defect case is much less likely to happen than the non-defect case. The defect class is thus the minority. The recognition of this class is more important, due to the failure of finding a defect could degrade software quality greatly. The challenge of learning from imbalanced data is that the relatively or absolutely underrepresented class cannot cause equal attention to the learning algorithm compared to the majority class, which often results in very specific classification rules or missing rules for the minority class without much generalization ability for the future prediction [9]. How to better distinguish data from the minority class is a major research question in class imbalance learning. Numerous methods have been proposed to tackle class imbalance problems at data and algorithm levels. Data-level approaches include a variety of resampling techniques, manipulating train data to amend the skewed class distributions, such as random oversampling, random undersampling, and SMOTE [10]. They are simple and effective, but their efficiency relies greatly on the problem and training algorithms [11].

Algorithm-level approaches dispose class imbalance by modifying their training mechanism directly with the idea of better accuracy on the minority class, containing one-class learning [12], and cost-sensitive learning algorithms [6], decision trees [13], neural networks [7], Naive Bayes [1], support vector machines [14], and Artificial Immune Systems [2]. Algorithm-level methods need specific treatments for different kinds of learning algorithms, which hinders their use in a lot of applications, because we do not know in advance which algorithm would be the best choice in most cases. Moreover, they didn't consider the data characteristic of class imbalance.

In addition to the data-level and algorithm-level solutions, Some researchers have noticed that the imbalanced distribution between defect and non-defect classes could degrade the performance of predictor highly, and attempted to leverage class imbalance learning techniques to reduce this negative effect. Menzies et al. [5] undersampled the non-defect class to balance train data, and checked how little information was required to learn a defect predictor. They

found that throwing away data does not degrade the performance of Naive Bayes and C4.5 decision trees. Some other studies also showed the helpfulness of resampling based on different learners [8], [15]. Ensemble algorithms and their cost-sensitive variants were investigated, and shown to be beneficial if a proper cost ratio can be set [7].

However, these studies all have no executed a comprehensive comparison among different class imbalance learning algorithms for SDP. It is still unclear in which aspect and to what extent class imbalance learning can benefit SDP problems, and which class imbalance learning approaches are more efficient. Such information would help us to understand the potential of class imbalance learning methods in this specific learning task, and develop better solutions. Thus, we will study a novel method of class imbalance learning in terms of how it facilitates SDP, and how it can be used better to solve SDP more efficiently through experiments and comprehensive analyses.

III. SOFTWARE DEFECT PREDICTION

This section introduces the two focal points of this paper. First, we describe how to leverage undersampling technique to balance data set. Subsequently, our solution of using CRF model and the work flow are described in detail.

A. Leverage Undersampling Technique To Balance Data Set

MeanShift algorithm is a nonparametric clustering technology, it does not require prior know the number of categories, also there is no limit to the shape of the cluster.

Given a train set t_1 , after segmentation, we can obtain majority class maj_1 and minority class min_1 . To begin with, for the majority class maj_1 , we utilize meanshift algorithm to cluster it into several classes, namely $\{c_1, c_2, \dots, c_n\}$. Secondly, in order to build a balance train set, We select samples randomly from each class according to the proportion to obtain a new majority class maj_2 . After that, the number of samples in maj_2 is approximately equal to min_1 . Therefore, the maj_2 and min_1 will be get together to structure a new train set t_2 which is balanced. In our UCRF method, we use the balanced train set t_2 to replace the original one and then apply CRF model in it.

B. Conditional Random Field

Conditional random field (CRF) [16] is conditional probability distributions that factorize according to an undirected model. The Linear-chain CRF model [16] may be the most widely used and simplest model of CRF models, in which a first-order Markov assumption is made on the hidden variables. A graphical model for this is shown in Figure 1.

In Figure 1, X is a set of input variables that are observed, such as a sequence of samples, and each variable X_i is a sample in software defect prediction. Y is a set of output variables that we want to predict, such as a sequence of sample labels in s software defect prediction, and each variable Y_i is a sample label for the corresponding sample X_i .

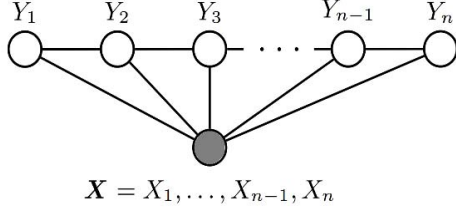


Figure 1 Graphical representation of Linear-Chain CRF

Under the condition that the value of random variable X is x , the conditional probability of the value of random variable Y is y can define as below.

$$P(y|x) = \frac{1}{Z(x)} \exp\{\sum_{k=1}^L \lambda_k f_k(y, x)\} \quad (1)$$

and

$$Z(x) = \sum_y \exp\{\sum_{k=1}^L \lambda_k f_k(y, x)\} \quad (2)$$

where $\{f_k\}$ are the features which are either state function or transition function, $\{\lambda_k\}$ are the weights of features, and $Z(x)$ is the partition function. The features $\{f_k\}$ can be described as:

$$f_k(y, x) = \sum_{i=1}^n f_k(y_{i-1}, y_i, x, i) \quad , \quad k=1,2,\dots,L \quad (3)$$

Moreover, in this figure we show $X = (X_1, \dots, X_n)$ as a single large observed node on which all of the factors depend, rather than showing each of the $X_1, \dots, X_n$ as individual nodes. To indicate in the definition of linear-chain CRF that each feature function can depend on observations from any time step, we have written the observation argument to f_k as a vector X_i , which should be understood as containing all the components of the global observations X that are needed for computing features at time i .

Although it encompasses HMM(Hidden Markov Model)-like models, the class of conditional random fields is much more expressive, because it allows arbitrary dependencies on the observation sequence. In addition, the features do not need to specify completely a state or observation, so that the model can be estimated from less train data. The other attractive property is the convexity of the loss function; indeed, CRF share all of the convexity properties of general maximum entropy models.

C. Workflow of Software Defect Prediction

In this paper, the work of Software defect prediction is viewed as a sequence labeling task by labeling each sample as Y or N. Y denotes the sample which has defect, N denotes the sample which has non-defect. Our workflow is summarized in Figure 2, and the detail is as follows:

1) *Undersample*: We implement meanshift algorithm by using the scikit-learn toolkit¹.

2) *Training the linear-chain CRF model* : Input the feature vectors of training set and L-BFGS algorithm is applied to estimate weight of each feature.

3) *Getting labels sequence*: Input the feature vector of a test question into the trained model, loopy belief propagation algorithm is applied to find the appropriate labels sequence of the test question.

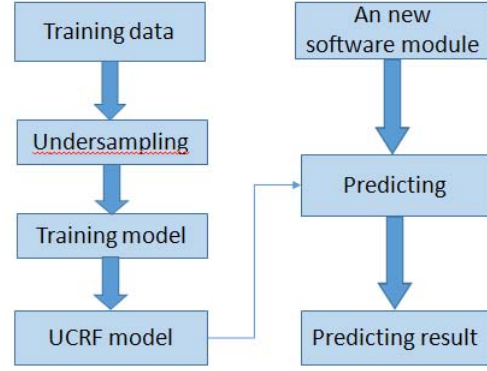


Figure2 Workflow of software defect prediction

IV. EXPERIMS AND ANALYSIS

A. Experimental Setup

This paper focuses on predicting the defect in Software module. In order to verify the performance of the proposed method, two experiments are designed as: (1) An experiment for comparing the performance of the UCRF method and CRF approach and RCRF approach. The CRF approach is to apply applying original train data in CRF model without undersample and the RCRF approach is to apply CRF model on the train data with random undersample. This experiment shows the contribution of our undersample technology. (2) An experiment for comparing the performance of the proposed method and the SVM(Support Vector Machine) based hybrid approach and BP neural network with undersample, this experiment shows the contribution of the CRF model.

The experiment results are calculated basedon the performanc e of 5-floed cross validation. In order to more directly and fairly compare our results with that of other methods, the Probability of Detection (PD), and the Probability of Non-defect (PN) are usually utilized as a performance measure. PD, also called recall, is the percentage of defective modules that are classified correctly within the defect class. PN ,also called recall, is the percentage of non-defective modules that are classified correctly within the non-defective class. For more comprehensive evaluation of predictors in the imbalanced context, G-mean is frequently utilized to measure how well the predictor can balance the performance between two classes. A good predictor should have high accuracies on both classes, which leads to a high G-mean. In the SDP context, $G\text{-mean} = \sqrt{PD * PN}$.

¹ Available at <http://scikit-learn.org/stable/>

Table 2. The result of the UCRF and the CRF and RCRF approach.

data	G-mean		
	SVM	BP neural network	UCRF
MC2	0.615	0.630	0.960
KC1	0.573	0.581	0.983
PC4	0.625	0.650	0.997
PC3	0.620	0.623	0.997
CM1	0.417	0.521	0.975
KC3	0.694	0.613	0.985
MW1	0.375	0.422	0.979
PC1	0.532	0.522	0.931
PC5	0.534	0.549	0.997
MC1	0.635	0.673	0.995
PC2	0.400	0.510	1.000

Table 3. The result of the UCRF and the SVM and BP neural network approach with undersample.

data	CRF			RCRF			UCRF		
	PD	PN	G-mean	PD	PN	G-mean	PD	PN	G-mean
MC2	0.923	1.000	0.960	0.821	0.347	0.532	0.923	1.000	0.960
KC1	0.851	1.000	0.922	0.871	0.397	0.588	1.000	0.967	0.983
PC4	0.950	1.000	0.974	0.850	0.338	0.534	1.000	0.996	0.997
PC3	1.000	1.000	1.000	0.839	0.338	0.537	1.000	0.995	0.997
CM1	0.750	1.000	0.866	0.861	0.377	0.570	1.000	0.952	0.975
KC3	0.000	1.000	0.000	0.667	0.661	0.664	1.000	0.972	0.985
MW1	0.000	1.000	0.000	0.875	0.340	0.545	1.000	0.960	0.979
PC1	0.000	1.000	0.000	0.911	0.334	0.548	0.867	1.000	0.931
PC5	0.960	1.000	0.979	0.858	0.401	0.586	1.000	0.996	0.997
MC1	0.000	1.000	0.000	0.879	0.339	0.545	1.000	0.992	0.995
PC2	0.000	1.000	0.000	0.600	0.667	0.632	1.000	1.000	1.000

In the experiments, the Linear-chain CRF model in the proposed method is implemented using MALLET². It uses a quasi-Newton method called L-BFGS [17] to find these feature weights efficiently and loopy belief propagation algorithm [18] to perform approximate inference. The SVM approach is implemented using LibSVM³.

B. Data Set

The experiment in this paper is done on the data set proposed by the public NASA MDP repository [19], which was also used by many studies. Thus, there are 11 data sets in total.

Table 1. The detail of data set

data	examples	attributes	Defect%
MC2	161	39	32.29
KC1	2109	21	15.45
PC4	1458	37	12.20
PC3	1563	37	10.23
CM1	498	21	9.38
KC3	458	39	9.38
MW1	403	37	7.69
PC1	1109	21	6.94
PC5	17186	39	3.00
MC1	9466	68	0.72
PC2	5589	38	0.41

Table 1 provides some basic summary information. Each data set is comprised of a number of software modules, each containing the corresponding number of defects and various software static code attributes. After preprocessing, modules that contain one or more defects were labeled as defective.

C. Results and analysis

From the results in Table 2, comparing the results of UCRF approach with that of CRF approach, there is a remarkable improvement after leveraging our undersample method to balance train data. It means that our undersample method is effective and beneficial. Moreover, from Table 2, we can see that the performance of UCRF is better than that of RCRF almost 4 percent. This proves that our undersample method which utilize meanshift algorithm to reduce majority class samples can decrease Redundant sample but no lost much imformation of original data set.

As shown in Table 3, compared to SVM approach and BP neural network approach, the UCRF approach boosts the G-mean by almost 3 percent in the SDP task. That is to say, it is effective to apply CRF model for SDP task and CRF model is more suitable for application in the train data which is after undersampling. Due to the features do not need to specify completely a state or observation, the model can be estimated from less train data. Moreover, the other attractive property of the CRF model is the convexity of the loss function and it actually share all of the convexity properties of general maximum entropy models. Thus, because of avoiding explicit modeling of the observed data, the proposed method can

² Available at <http://mallet.cs.umass.edu/>

³ Available at <http://www.csie.ntu.edu.tw/~cjlin/libsvm/>

incorporate the classification of software defect data with different statistics characteristics into a unified probabilistic framework.

V. CONCLUSION

In this paper, we have presented a novel framework for software defect prediction. The framework involves Underdsample technique and prediction. In the Underdsample stage, we take advantage of meanshift clustering method to reduce the samples of majority class for balancing the train data set. Then, in the prediction stage, it is highly effective and valuable to apply the CRF model in the above balanced train data set. Experiment result shows that the proposed method UCRF yields an excellent performance for software defect prediction, in comparisons of the state-of-the-art approach.

ACKNOWLEDGMENT

The work was supported by Postdoctoral Science Foundation of China (No. 2014M552171).

REFERENCES

- [1] T. Menzies, J. Greenwald, and A. Frank, "Data mining static code attributes to learn defect predictors," *IEEE Trans. Software Eng.*, vol. 33, no. 1, pp. 2–13, Jan. 2007.
- [2] C. Catal and B. Diri, "Investigating the effect of dataset size, metrics sets, and feature selection techniques on software fault prediction problem," *Inf. Sci.*, vol. 179, no. 8, pp. 1040–1058, 2009.
- [3] Y. Ma, L. Guo, and B. Cukic, "A statistical framework for the prediction of fault-proneness," *Adv. Mach. Learn. Appl. Softwre Eng.*, pp. 237–265, 2006.
- [4] T. Hall, S. Beecham, D. Bowes, D. Gray, and S. Counsell, "A systematic review of fault prediction performance in software engineering," *IEEE Trans. Software Eng.*, vol. 38, no. 6, pp. 1276–1304, Nov.-Dec. 2012.
- [5] T. Menzies, B. Turhan, A. Bener, G. Gay, B. Cukic, and Y. Jiang, "Implications of ceiling effects in defect predictors," in *Proc. 4th Int. Workshop PredictorModels Software Eng. (PROMISE 08)*, 2008, pp. 47–54.
- [6] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Trans. Knowledge Data Eng.*, vol. 21, no. 9, pp. 1263–1284, Sep. 2009.
- [7] J. Zheng, "Cost-sensitive boosting neural networks for software defect prediction," *Expert Syst. Appl.*, vol. 37, no. 6, pp. 4537–4543, 2010.
- [8] J. C. Riquelme, R. Ruiz, D. Rodriguez, and J. Moreno, "Finding defective modules from highly unbalanced datasets," *Actas de los Talleres de las Jornadas de Ingeniería del Software y Bases de Datos*, vol. 2, no. 1, pp. 67–74, 2008.
- [9] G. M. Weiss, "Mining with rarity: A unifying framework," *ACM SIGKDD Explor. Newslett.*, vol. 6, no. 1, pp. 7–19, 2004.
- [10] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 341–378, 2002.
- [11] A. Estabrooks, T. Jo, and N. Japkowicz, "A multiple resampling method for learning from imbalanced data sets," *Comput. Intell.* 20, vol. 20, no. 1, pp. 18–36, 2004.
- [12] N. Japkowicz, C. Myers, and M. A. Gluck, "A novelty detection approach to classification," in *Proc. IJCAI*, 1995, pp. 518–523.
- [13] T. M. Khoshgoftaar and N. Seliya, "Tree-based software quality estimation models for fault prediction," in *Proc. 8th IEEE Symp. Software Metrics*, 2002, pp. 203–214.
- [14] D. Gray, D. Bowes, N. Davey, Y. Sun, and B. Christianson, "Using the support vector machine as a classification method for software defect prediction with static code metrics," *Eng. Appl. Neural Netw.*, vol. 43, pp. 223–234, 2009.
- [15] L. Pelayo and S. Dick, "Evaluating stratification alternatives to improve software defect prediction," *IEEE Trans. Rel.*, vol. 61, no. 2, pp. 516–525, Jun. 2012.
- [16] Lafferty, J., McCallum, A., & Pereira, F. Conditional random fields: Probabilistic models for segmenting and Labeling Sequence Data. *ICML '01 Proceedings of the Eighteenth International Conference on Machine Learning*. pp: 282-289. 2001.
- [17] Sha, F., & Pereira, F. Shallow parsing with conditional random fields. In *Proceedings of the 2003 Conference of the North American Chapter of the Association for Computational Linguistics on Human Language Technology*. Volume 1, pp: 134-141. 2003.
- [18] Taskar, B., Abbeel, P., & Koller, D. Discriminative probabilistic models for relational data. In *Proceedings of the Eighteenth conference on Uncertainty in artificial intelligence*. pp: 485-492. 2002.
- [19] M. Chapman, P. Callis, and W. Jackson, "Metrics Data Program," technical report, NASA IV and V Facility, 2004.